FINAL YEAR B.TECH PROJECT

IoT-Based Predictive Maintenance System

for Packaging Line Optimization

Mondelez International Case Study — OEE Recovery Framework

Rohan Kumar | Roll: 2311601321
Tushar Singh • Devyanshu Prakash • Aditya Sen
Dept. of Mechanical Engineering | MANIT Bhopal
Mentor: Dr. Rajesh Purohit

OEE Drop

98%→72%

IoT Solution

ESP32 + Cloud

Target Recovery

90%+ OEE

Problem Statement

Mondelez International — Packaging Line 2 Crisis

BEFORE

98%

OEE — Packaging Line 2

→

AFTER — RECENT

72%

Erratic stoppages & breakdowns

LOSS

26pts

OEE points lost

Three Failing Components:

Photo-eye Sensors

Random failures — dust & voltage fluctuations

Random Failure

Sealing Jaws

Fail ~300 hrs — heat wear, temp at 143°C vs 140°C target

Wear-out

Servo Motors

Fail in first 40 hrs — infant mortality / mfg defects

Infant Mortality

⚠ **Current Strategy: Replace ALL components every 150 hrs — NOT WORKING. Costs rising, healthy parts being replaced.**

Project Objectives

What this system is designed to achieve

01

Real-Time Monitoring

Continuous tracking of temperature, vibration, acoustic & current signals from all three critical components

02

Edge Processing

ESP32 microcontroller reads sensors, applies threshold checks, and triggers instant local alerts (LED/buzzer)

03

Cloud Integration

MQTT/HTTP to ThingSpeak — automated time-series storage, dashboard visualization, and email alerts

04

Fault Classification

Classify: minor stop (<10 min) vs major breakdown (>10 min); Pareto chart identifies top OEE loss contributors

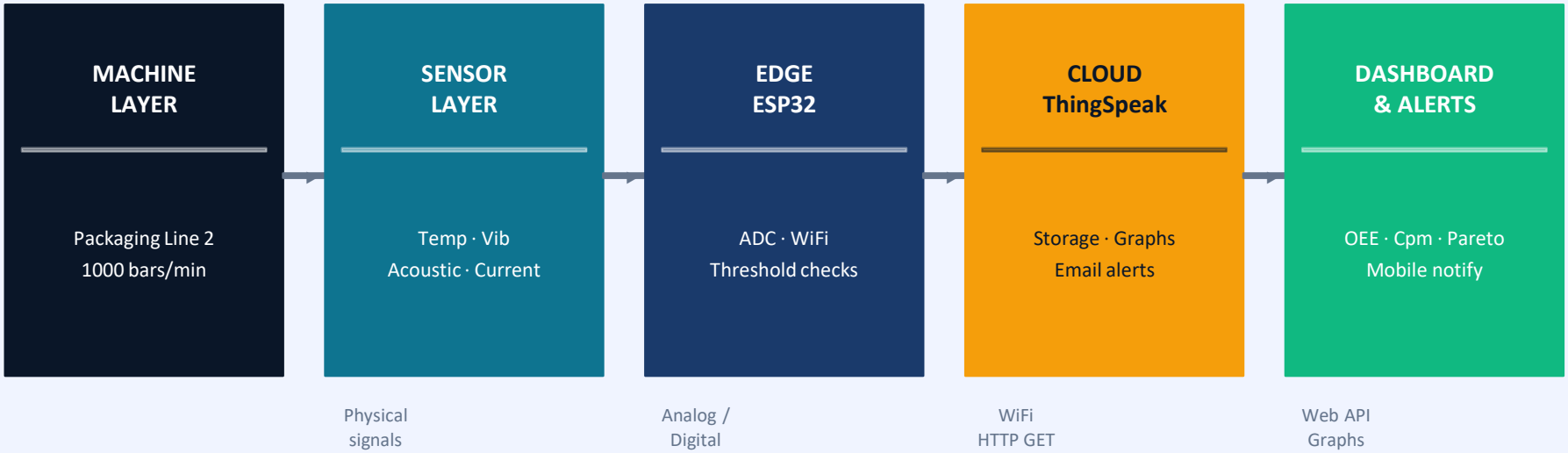
05

Process Capability

Real-time Cp, Cpk, Cpm calculation — alert when sealing jaw temperature drifts from 140°C target

System Architecture

End-to-end data pipeline — from sensors to decisions



Key Sensors Used:

- LM35 — Temperature (Sealing Jaw)
- SW-420 — Vibration (Servo Motor)
- MAX9814 — Acoustic (Motor Sound)
- ACS712 — Current (Motor Load)

Hardware Components & Cost

Student-level budget — full system under ₹1500

Component	Purpose	Specs	Cost (₹)
ESP32 DevKit	Main MCU + WiFi + MQTT	Dual-core, 12-bit ADC, 2.4GHz WiFi	₹400
LM35 Sensor	Sealing jaw temperature	10mV/°C, ±0.5°C, –55°C to +150°C	₹50
SW-420 Vibration Module	Motor vibration detection	Digital HIGH on vibration, 3.3–5V	₹50
MAX9814 Microphone	Acoustic / sound sensing	40–60dB, 10Hz–10kHz range	₹150
ACS712 (5A/30A)	Motor current monitoring	±5A or ±30A, 2.5V zero offset	₹80
LEDs + Buzzer	Local fault alerts	Standard 5mm LED + 3.3V buzzer	₹30
Jumper wires + Breadboard	Prototyping	Male-to-male & female jumpers	₹100
TOTAL		*All items available on Amazon/Robu.in*	**≈₹860–1200**

Connection Summary:

LM35 Vout → GPIO34 (ADC) | SW-420 D0 → GPIO27 (Digital) | ACS712 OUT → GPIO35 (ADC) | Buzzer → GPIO14

Circuit & Wiring Guide

Step-by-step connections for the prototype

STEP 1 — Power Setup

USB 5V → ESP32 VIN. Common GND to all sensors. Use 3.3V output pin for sensor VCC.

STEP 2 — LM35 (Temperature)

VCC→3.3V, GND→GND, Vout→GPIO34. Mount on jaw with thermal paste for real use.

STEP 3 — SW-420 (Vibration)

VCC→3.3V, GND→GND, D0→GPIO27. Mount on motor housing. Adjust sensitivity pot.

STEP 4 — ACS712 (Current)

Insert sensor in series with motor power line. Vout→GPIO35. Zero = 2.5V at 0 amps.

STEP 5 — LED + Buzzer (Alerts)

LED (+ 220Ω resistor) → GPIO14. Buzzer → GPIO13. Both trigger HIGH on fault.

STEP 6 — Upload Code

Arduino IDE → Board: ESP32 Dev Module. Fill WiFi SSID, ThingSpeak API key. Flash.

```
temp = analogRead(34) * (3.3/4095.0) * 100;   //  LM35: 10mV/°C  →  ADC to °C conversion
```

Software Stack & Code Logic

ESP32 Arduino code → ThingSpeak cloud API

ESP32 Program Flow:

setup()	WiFi.begin(ssid, password) → Connect to network
↓	
Read Sensors	analogRead(34) → LM35 temp°C digitalRead(27) → SW-420 vibration
↓	
Edge Checks	if temp > 140°C → digitalWrite(LED, HIGH) if vibState == HIGH → Serial alarm
↓	
ThingSpeak POST	GET /update?api_key=XXX&field1=temp&field2=vib
↓	
delay(15000)	ThingSpeak rate limit: max 1 update per 15 sec

Tools: Arduino IDE + Python + ThingSpeak + Streamlit

ESP32 Arduino Code (Key Snippet)

```
void loop() {
  int rawT = analogRead(34);
  float tempC = rawT*(3.3/4095.0)*100;
  int vib = digitalRead(27);

  // Local alert
  if (tempC > 140) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("OVERHEAT!");
  }

  // Send to ThingSpeak
  WiFiClient c;
  if (c.connect(server, 80)) {
    String url = "/update?api_key="
      + apiKey + "&field1="
      + tempC + "&field2=" + vib;
    c.print("GET " + url +
      " HTTP/1.1\r\nHost: "
      + server + "\r\n\r\n");
  }
  delay(15000);
}
```

Cloud Setup — ThingSpeak

From device data to live dashboard in 5 steps

1

Create Account

Go to thingspeak.com → Sign up free. Supports up to 4 channels free tier.

2

New Channel

Channels → My Channels → New Channel. Name: "PackagingLineMonitor". Enable Field1=JawTemp, Field2=MotorVib, Field3=Current.

3

Get API Key

API Keys tab → Copy Write API Key. Paste into ESP32 code as apiKey variable.

4

Configure Alerts

Apps → ThingSpeak React → New React: "If JawTemp > 140, send email". Add for MotorVib==1 too.

5

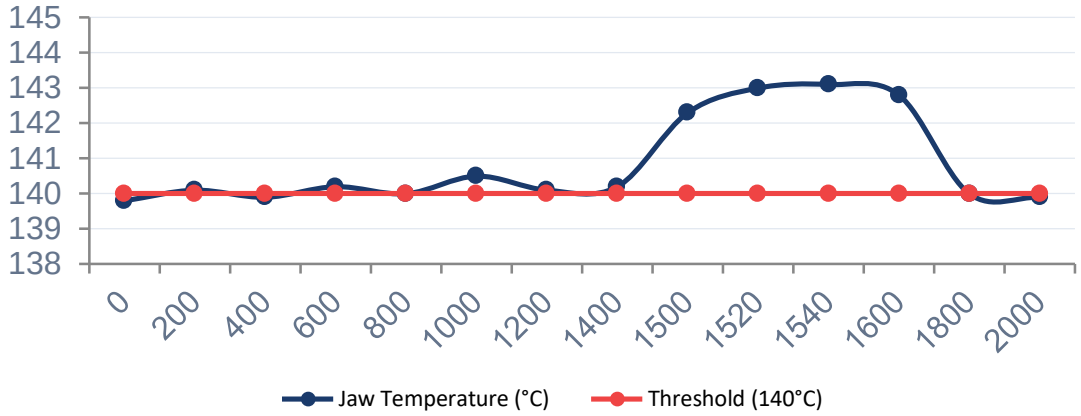
View Dashboard

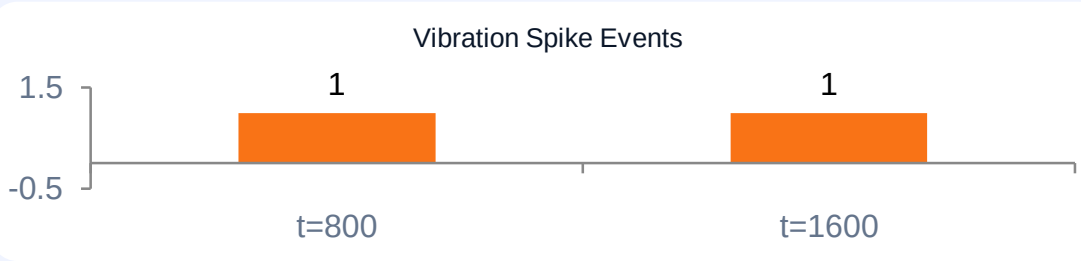
Private View shows live graphs. Or use MATLAB Visualizations for combined Cp/Cpk charts and trend analysis.

Alternative: MQTT (Mosquitto broker) → Raspberry Pi → InfluxDB + Grafana for advanced deployment

Simulated Data & Results

System detects overheat and vibration faults early





System Detects:

Overheat Warning

Jaw temp → 143°C at t=1500 min detected 50 min before breakdown

Vibration Alert

Motor spikes at t=800 and t=1600 — bearing wear caught early

OEE Recovery

Preventing these events: +8–14 OEE points (literature)

Reliability Engineering

Bathtub Curve mapping + Process Capability Analysis

Bathtub Curve — Component Mapping:

Servo Motors (Infant Mortality)	Photo-eye Sensors (Random Failure)	Sealing Jaws (Wear-out)
Fail in first 40 hours	Unpredictable — throughout life	Fail around 300 hours

✗ Why 150-hr Strategy Fails:

Motors fail at 40 hrs — waiting 150 hrs means 3 failures before any check. Jaws fail at 300 hrs — replacing at 150 hrs doubles cost with no benefit. Sensors have random failure — no time pattern to exploit.

Process Capability — Sealing Jaw Temp:

`Cp = (145−135)/(6×1) =`	1.67 ✓ Capable spread
`Cpk = min[2/3, 8/3] =`	0.67 ⚠ Off-center
`Cpm = 10/6√(1+9) =`	0.527 ✗ Far from target

Key: Operating at 143°C (target 140°C) causes thermal wear even within spec!

OEE Improvement Strategy

How IoT targets each OEE component

OEE = Availability × Performance × Quality

Availability

Run Time / Planned Time

Issue: Unplanned breakdowns

Fix: Predictive alerts prevent failures → less MTTR

Before: ~87%

After: 95%+

Performance

Actual / Theoretical Output

Issue: Minor stops untracked

Fix: Auto-log all stops → Pareto top 3 → eliminate

Before: ~88%

After: 95%+

Quality

Good Units / Total Units

Issue: Bad seals from 143°C deviation

Fix: Cpm alert catches temp drift before defects occur

Before: ~94%

After: 99%+

Overall: 72% (current) → 90%+ (projected) | Downtime reduction: ~40% | Spare part savings: cut unnecessary 150-hr replacements

Conclusion & Future Scope

What we built — and where it goes next

✓ What Was Achieved:

- Real-time monitoring of temperature (LM35), vibration (SW-420), and current (ACS712) on all 3 critical components
- Automatic fault detection: overheat >140°C and vibration spike → instant local LED+buzzer alert
- ThingSpeak cloud dashboard with live graphs, email alerts, and MATLAB analysis capability
- Bathtub-curve–driven maintenance strategy: right approach for each component failure mode
- Process capability (Cpm = 0.527) reveals thermal damage from 143°C operation — not visible from Cpk alone
- Projected OEE improvement: 72% → 90%+ | Downtime cut: ~40% | Strategy: condition-based, not 150-hr blanket

Future Enhancements:

ML / LSTM

Train on historical data to predict RUL and failure probability

Mobile App

Blynk / Flutter push notifications for on-the-go monitoring

Digital Twin

MATLAB/Simulink model fed by live sensor data

Edge AI

TinyML on ESP32 — anomaly detection without internet

CMMS Link

Auto-create work orders when thresholds are breached

"This project transforms post-mortem fixes into preventive alerts — turning Industry 4.0 concepts into a real engineering solution."